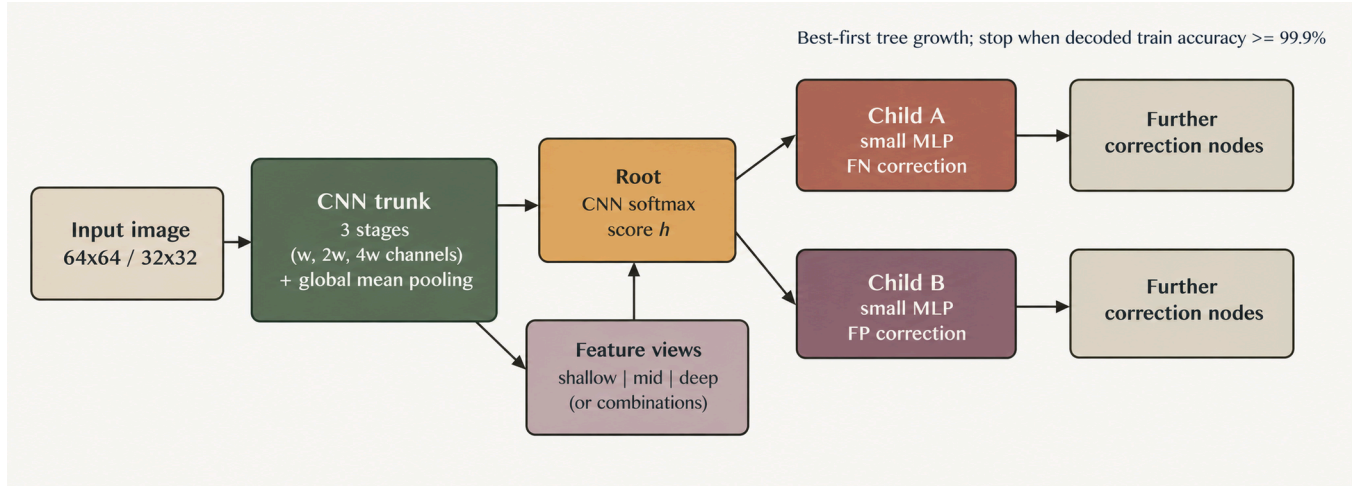


TREE-TYPE NEURAL NETWORK

Small neural-network nodes on a shared CNN feature trunk

Cats-vs-Dogs | CIFAR-10 | Overfitting | Double Descent | Receptive Fields | Size Minimisation



Final multiclass prediction: one tree per class (one-vs-rest), then argmax

Figure 1. The final design: a CNN trunk supplies multi-scale pooled feature views; the CNN softmax head acts as the root and small MLPs recursively correct false negatives/false positives.

99.99%	99.95%	3.9x	3 / 3
CD tree train accuracy	CIFAR-10 tree train accuracy	CD parameter reduction found	CIFAR-10 DD seed support

All numerical results reported below are taken directly from the completed experimental notebook; interpretations are grounded in the measured results.

Author: Arnav

Private GitHub repository: <Empty>

Executive Summary

Central idea

The experiment treats the CNN root as an intentionally under-fit classifier and makes the tree responsible for the remaining training errors. Each correction node is a small MLP operating on selected multi-scale CNN feature views. The resulting system is a concrete implementation of a tree-type neural network whose capacity can be grown, probed, and pruned.

The story of the experiment is deliberately coherent. First, a compact CNN is trained only to about 90-95% training accuracy, creating a controlled residual-error regime. Second, the tree grows correction nodes until the decoded training accuracy reaches 99.9% or above. Third, the enlarged train-test gap, the train-vs-test correction counts, and the double-descent sweep make the requested overfitting behavior visible. Finally, occlusion maps inspect what the different nodes attend to, while parameter-count sweeps and post-hoc pruning ask how much of the resulting model is actually needed.

Assignment objectives at a glance

Objective	Evidence in notebook	Assessment
Tree with small neural-network nodes on both datasets	CNN root + MLP correction nodes; one-vs-rest ensemble for CIFAR-10; toy correctness test	Strongly met
Clever feature construction	Shared CNN trunk with shallow/mid/deep pooled views; node role/depth controls view selection	Strongly met
Overfit to near 100% train accuracy	99.99% CD; 99.95% CIFAR-10; explicit fixes/breaks analysis	Strongly met
Try double descent after interpolation	20% noisy labels; width sweep; 3 seeds; clean test evaluation	Meaningful empirical evidence
Visualise receptive fields and summarise	Occlusion sensitivity maps for root/depth-1/deeper nodes; quantitative concentration/correlation statistics	Experiment met; written summary to be included
Minimise network size	Smaller trunks; node-size sweep; pruning; 1-s.e. validation rule	Strongly attempted

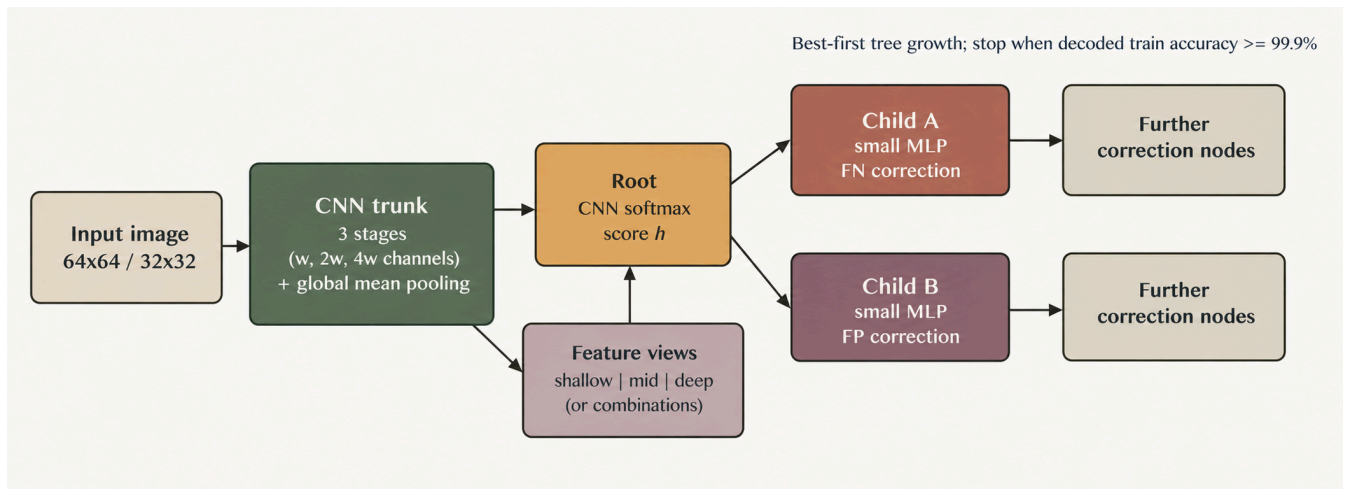
1. Architecture and Implementation

1.1 Problem setup and tree mechanism

At each tree node, the parent produces a scalar score h . For a binary target, the training set is partitioned into true negatives, true positives, false negatives, and false positives under the parent hard decision. Child A is trained on the false-negative correction task while ignoring the parent's true positives; Child B is trained on the false-positive correction task while ignoring the parent's true negatives. The parent score is then corrected as

$$z = h + \alpha A - \beta B, \quad \alpha \geq 0, \beta \geq 0$$

The implementation grows the tree best-first: unresolved correction sets with larger error mass are prioritised, a child MLP is trained on the corresponding set, and the process continues until the decoded training accuracy reaches the 0.999 target or a budget is exhausted. The alpha/beta weights are fitted on the node training sets, with the candidate value 0 included so that a correction is never forced to make the parent worse on that fitting set.



Final multiclass prediction: one tree per class (one-vs-rest), then argmax

Figure 2. Architecture used in the final experiments. The CNN trunk and its softmax root are trained together; child MLPs are subsequently trained on frozen, standardised trunk features.

Important implementation distinction. The CNN trunk and its softmax root are trained end-to-end as the base model. The child MLPs are then trained on extracted trunk features; the full tree is not jointly backpropagated through the CNN trunk in the same optimisation loop.

1.2 CNN trunk and feature views

Component	Implementation
Input	Cats-vs-Dogs resized to 64x64; CIFAR-10 kept at 32x32.
Stem / stage 1	Two 3x3 Conv-BN-ReLU blocks, first downsampling on 64x64 inputs, followed by 2x2 max-pooling.
Stage 2	Two Conv-BN-ReLU blocks, channel width 2w, followed by 2x2 max-pooling.
Stage 3	Two Conv-BN-ReLU blocks, channel width 4w.
Feature views	Global mean pooling exposes shallow, mid, and deep vectors with dimensions w, 2w, and 4w; concatenated views are also available.
Root	Linear head on the deepest pooled feature. Binary root uses a log-odds score; CIFAR-10 uses ten one-vs-rest log-odds scores.
Feature standardisation	Extracted features are standardised using training-set mean/std only, then clipped to a bounded range.

Selected feature strategy: the final model uses the varied strategy. Depth-1 A nodes receive mid+deep features, depth-1 B nodes receive deep features, and deeper nodes rotate among shallow+mid, mid+deep, and deep views. This is the notebook's main innovation over simply giving every node one identical feature vector.

Child-node detail	Implementation
MLP shape	Small fully connected MLP with ReLU hidden layers; two-hidden-layer selected model uses widths [16, 32].
Training	Adam, learning rate 1e-2, up to 60 epochs in the main tree experiments; tiny hard sets are given at least 300 optimisation steps.
Loss	BCEWithLogitsLoss with capped positive re-weighting to handle imbalanced correction sets.
Pure correction sets	Constant nodes are used when a correction set contains only one target value.
Feature routing	Role/depth-dependent routing in the varied strategy; a shared strategy using all pooled views is used as a comparison.

1.3 Data and model-selection protocol

The final notebook uses a deliberately manageable subset while keeping a held-out validation split and a separate test set. The validation split is used for hyper-parameter and configuration selection; the test set is not used for these choices.

Dataset	Training pool	Train	Validation	Test
Cats-vs-Dogs	8,000	6,800	1,200	2,000
CIFAR-10	15,000	12,750	2,250	10,000

Selection logic

The base CNN is selected from a width/optimisation/epoch grid subject to 90-95% TRAIN accuracy and best VALIDATION accuracy among in-band candidates. The final tree configuration is then selected using a one-standard-error rule on validation accuracy among configurations that reach the 99.9% training target.

2. Detailed Results

2.1 Base CNN is intentionally under-fit

Dataset	Model	Train	Val	Test
Cats-vs-Dogs	Converged CNN, 30 ep	97.93%	90.75%	90.55%
Cats-vs-Dogs	Selected under-fit CNN, w=64, 16 ep	93.81%	89.58%	88.75%
CIFAR-10	Converged CNN, 30 ep	99.00%	84.58%	85.04%
CIFAR-10	Selected under-fit CNN, w=64, 16 ep	92.82%	84.09%	82.90%

This creates a controlled setting in which the CNN is useful but still leaves hundreds of training errors for the tree to correct: 421 errors on Cats-vs-Dogs and 915 errors on CIFAR-10.

2.2 Final CNN + tree models

Dataset	Strategy	Tree depth	Nodes	Tree params	Train	Val / Test
Cats-vs-Dogs	varied, [16,32]	3	5	22,858	99.99%	88.17% / 87.20%
CIFAR-10	varied, [16,32]	3	40	160,166	99.95%	83.11% / 81.70%

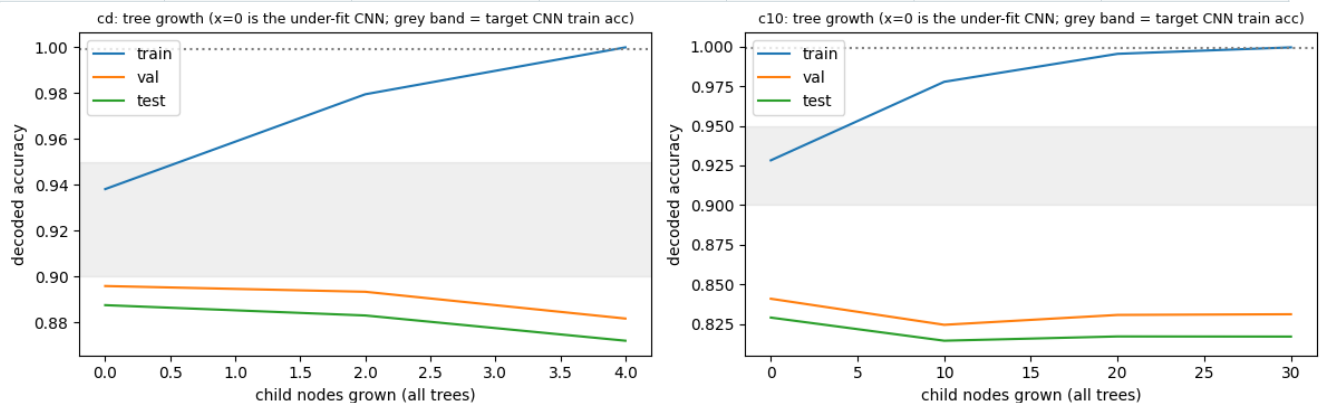


Figure 3. Tree-growth curves. The root starts at the under-fit CNN accuracy, while adding correction nodes drives the decoded training accuracy to approximately 100%. Validation/test accuracy decline rather than improve, making the requested overfitting visible.

2.3 What the tree actually changes

Dataset	Split	CNN errors fixed	CNN errors broken	Net change
Cats-vs-Dogs	Train	420	0	+420
Cats-vs-Dogs	Test	52	83	-31
CIFAR-10	Train	913	4	+909
CIFAR-10	Test	239	359	-120

On the training set, the correction mechanism is strongly asymmetric in the intended direction: it fixes almost all of the root's mistakes while breaking very few. On clean test data, the balance reverses, which is direct evidence that the learned corrections have overfit the training residuals rather than providing a generalisation improvement.

2.4 Overfitting is explicit, not incidental

Dataset	Train gain	Test change	Train-test gap: CNN	Train-test gap: tree
Cats-vs-Dogs	+6.18 pts	-1.55 pts	5.1 pts	12.8 pts
CIFAR-10	+7.13 pts	-1.20 pts	9.9 pts	18.3 pts

Interpretation supported by the numbers

The requested overfitting regime is clearly present: the tree spends extra capacity to interpolate the training set, but the resulting increase in training accuracy is accompanied by a larger train-test gap and lower test accuracy.

2.5 Double-Descent Experiment

The double-descent experiment injects 20% label noise into a training subset, keeps evaluation labels clean, sweeps model width, and repeats each width over three random seeds. The tree and trunk widths increase together; the plots report both noisy-train error and clean-test error against total trainable parameter count.

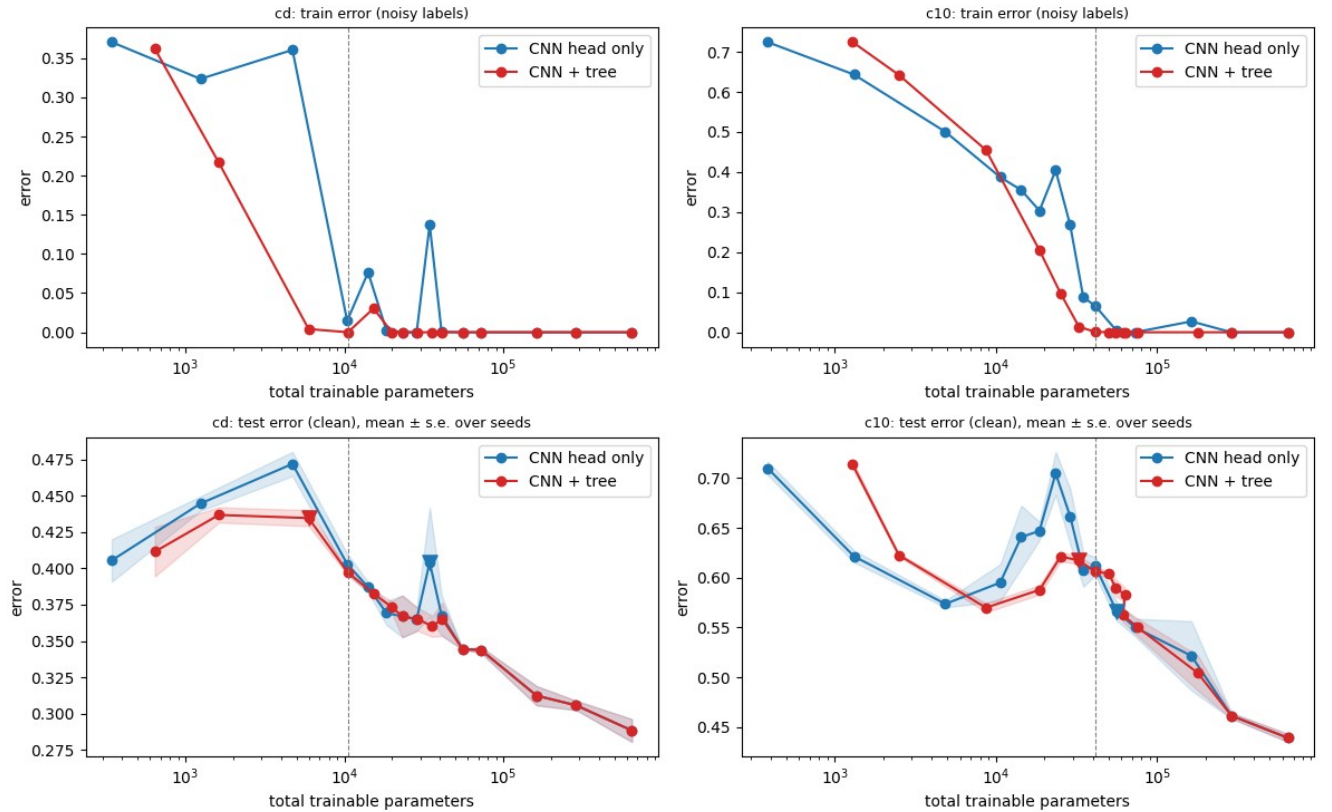


Figure 4. Double-descent sweep. The lower panels show clean-test error. The dashed vertical marker identifies the first width at which the noisy training accuracy reaches the interpolation threshold. The CNN+tree curves show a hump around the interpolation region followed by declining test error at larger model sizes.

Dataset	CNN only	CNN + tree	Tree peak	Seeds supporting tree pattern
Cats-vs-Dogs	Suggestive; inconsistent	Supported / heuristic	w=4	2/3
CIFAR-10	No convincing double descent	Supported / heuristic	w=8	3/3

For Cats-vs-Dogs, the mean tree test error rises by about 0.023 around the peak and later drops by about 0.146; two of three seeds support the same rise-and-drop pattern. For CIFAR-10, the corresponding rise is about 0.047 and the later drop about 0.178, with all three seeds supporting the pattern.

Double-descent interpretation

The sweep provides empirical evidence consistent with double descent around the interpolation regime, especially for CNN+tree. Because the conclusion comes from a finite width sweep, three seeds, and a heuristic peak-detection rule, it should be presented as evidence rather than as a definitive characterisation of double descent.

2.6 Receptive-Field / Input-Sensitivity Analysis

The notebook uses occlusion sensitivity. An 8x8 grid is overlaid on each image; each cell is occluded in turn, and the absolute change in the node score is measured. The resulting map indicates which spatial regions have the largest effect on the node score.

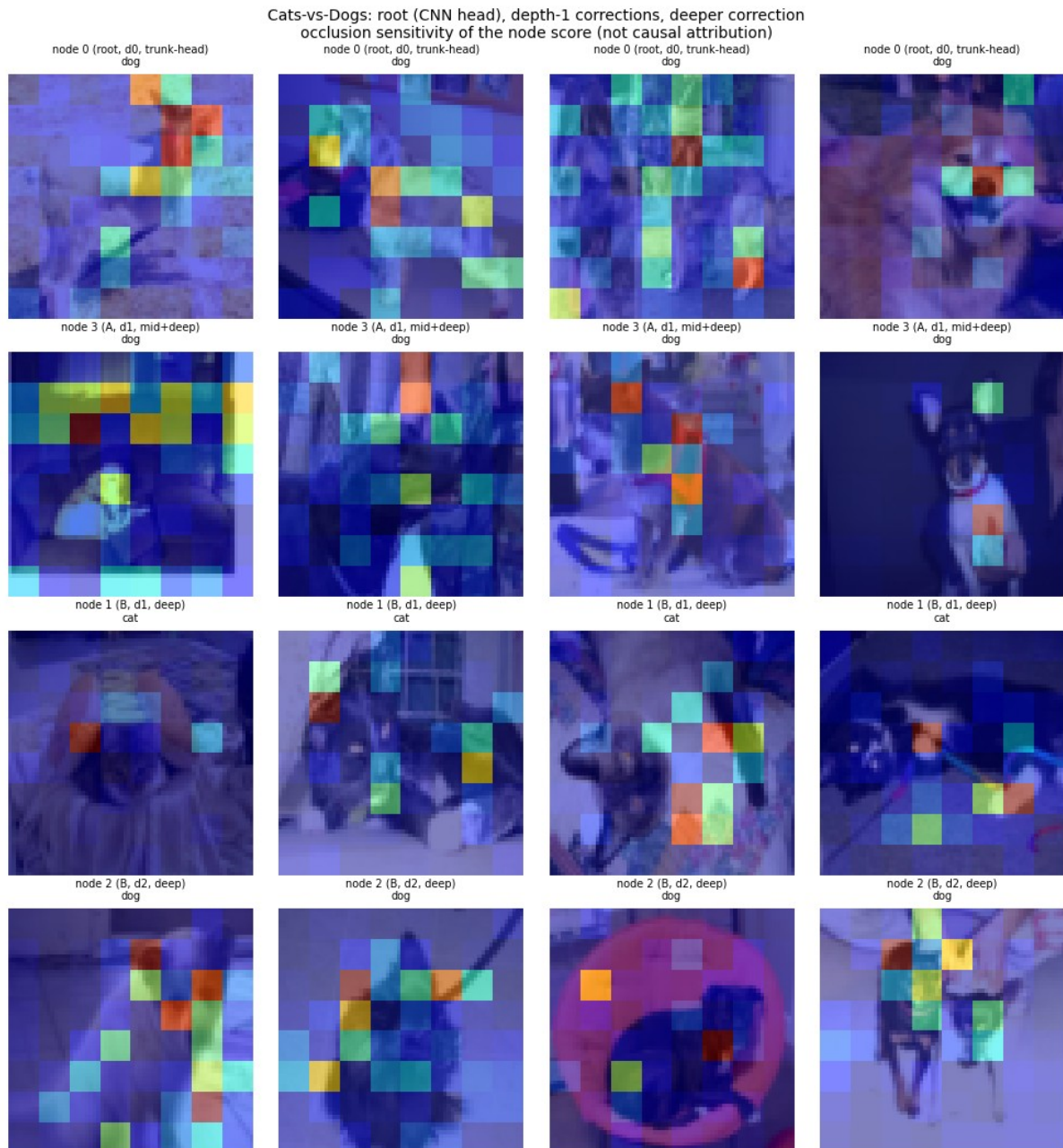


Figure 5. Cats-vs-Dogs occlusion maps for the root, depth-1 correction nodes, and a deeper correction node. Bright regions indicate larger changes in the node score when the region is occluded.

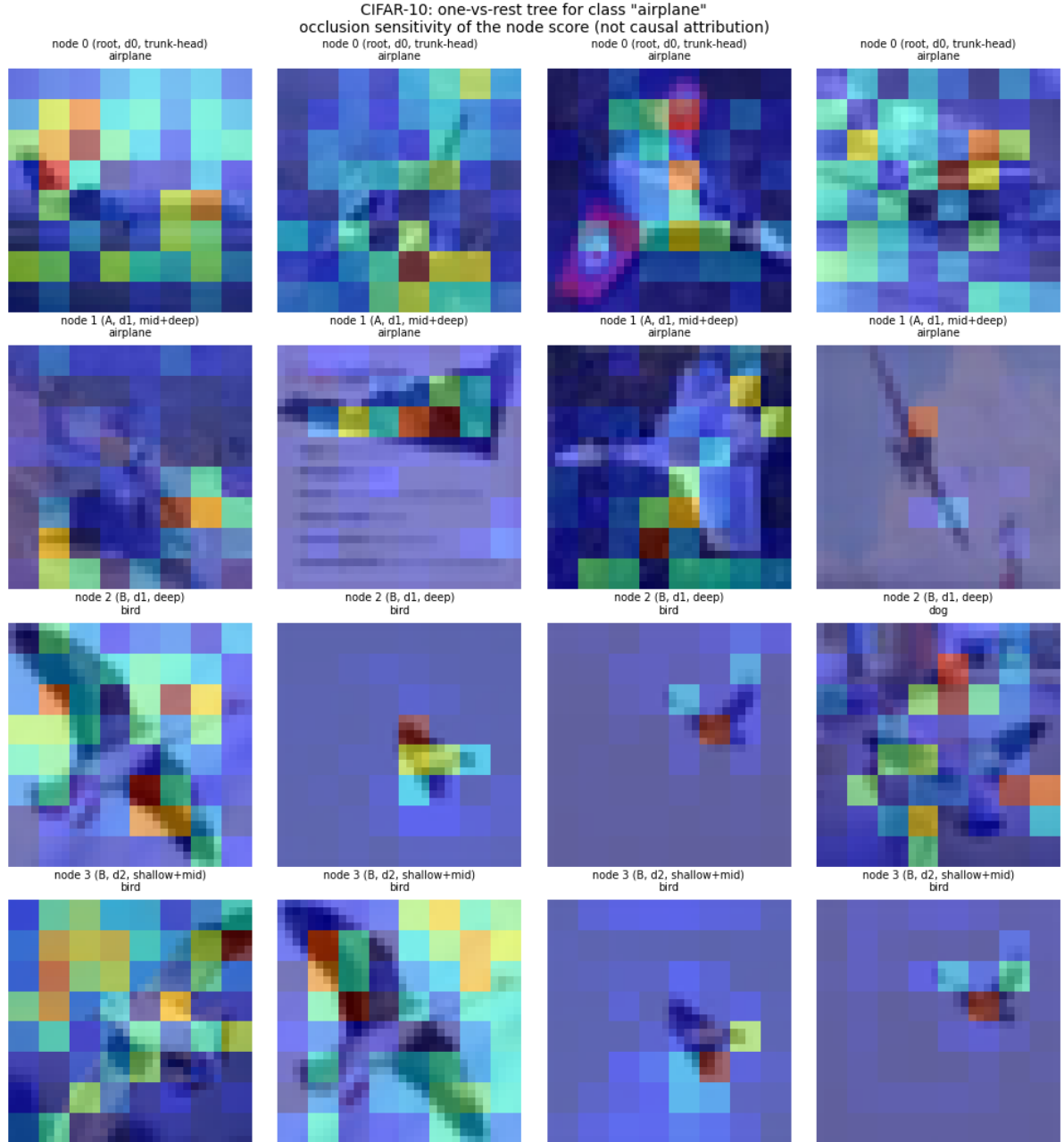


Figure 6. CIFAR-10 occlusion maps for a one-vs-rest airplane tree. Child nodes often display more concentrated or differently located sensitivity than the root.

Quantitative observations

Dataset / node	Top-10% sensitivity mass	Centre 4x4 mass	Correlation with root
CD root	0.359	0.361	-
CD A, depth 1	0.473	0.399	0.800
CD B, depth 1	0.548	0.408	0.676
CD B, depth 2	0.443	0.393	0.748
C10 root	0.280	0.320	-
C10 A, depth 1	0.513	0.470	0.446
C10 B, depth 1	0.500	0.470	0.610
C10 B, depth 2	0.393	0.355	0.453

Across the representative nodes/images sampled by the notebook, the mean mass in the most sensitive 10% of spatial cells is 0.46 for Cats-vs-Dogs and 0.42 for CIFAR-10, compared with a uniform baseline of 0.094. Mean child-vs-root map correlation is about 0.74 for Cats-vs-Dogs and 0.50 for CIFAR-10.

Correct interpretation

These are occlusion-based effective receptive-field / input-sensitivity maps, not exact theoretical receptive fields and not causal attribution maps. Because the CNN features are globally pooled, a strict architectural receptive field may encompass the whole image; the experiment instead shows where perturbations have the largest effect on each node score. The lower child/root correlation on CIFAR-10 is consistent with stronger feature specialisation away from the root in the sampled nodes.

2.7 Network Size Minimisation

Size is treated as a design variable rather than an afterthought. The notebook explores narrower CNN trunks, several child-MLP configurations, and post-hoc subtree pruning. A model is considered “good” only if it reaches the interpolation target and remains within one binomial standard error of the best validation accuracy among interpolating candidates.

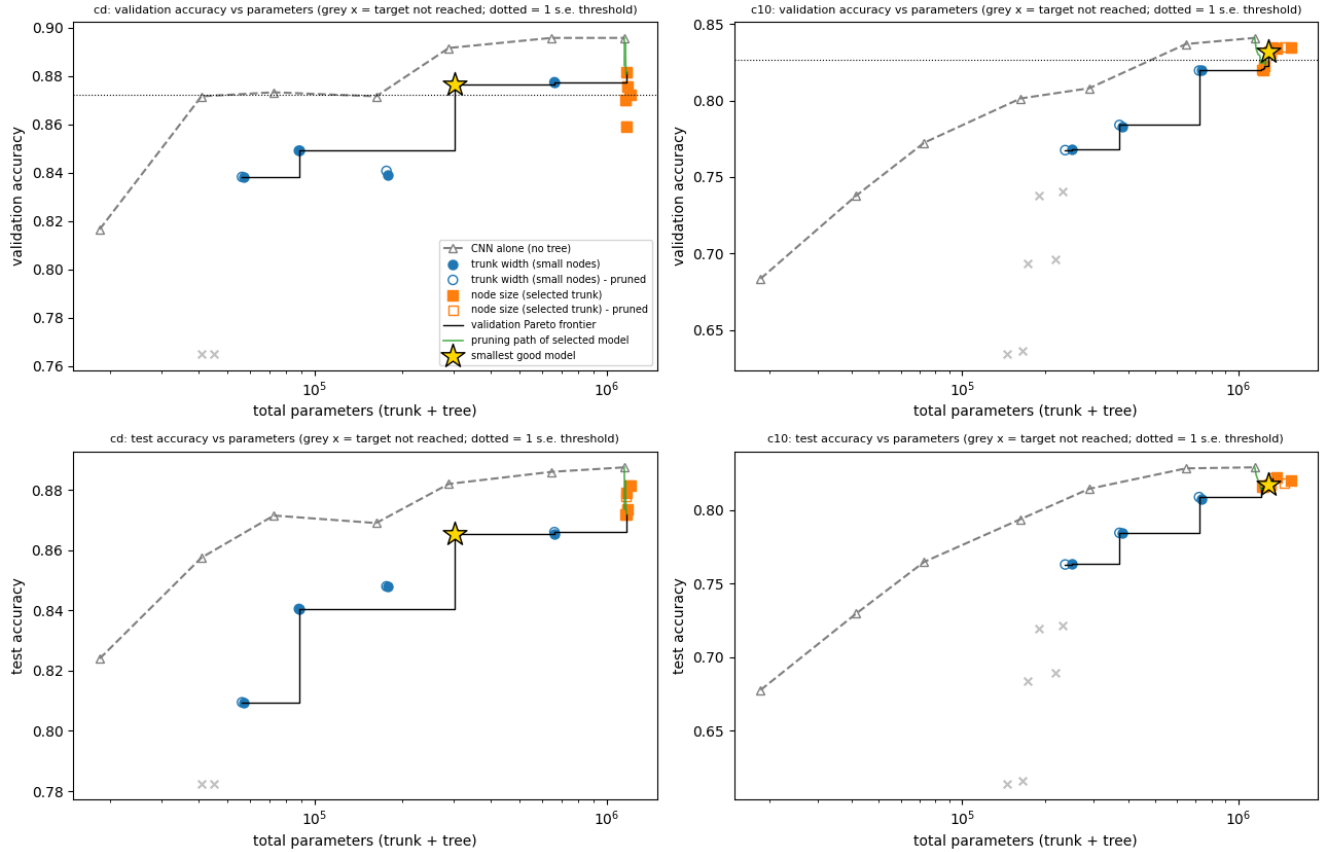


Figure 8. Validation/test accuracy versus total trainable parameters. The highlighted star marks the smallest good model found in the explored candidate space.

Dataset	Selected main model	Smallest good found	Tree nodes	Train / Val / Test	Reduction vs main
Cats-vs-Dogs	1,169,676 params	302,285 params	14	99.91% / 87.67% / 86.55%	74.2%
CIFAR-10	1,309,040 params	1,288,234 params	36	99.91% / 83.20% / 81.76%	1.6%

The Cats-vs-Dogs search produces a meaningful compression result: the smallest good model found uses a $w=32$ trunk, one-hidden-layer [8]-wide nodes, and pruning, reducing parameters by about 74% relative to the selected main model. CIFAR-10 is much less compressible within the explored candidates: the smallest good model found is only about 1.6% smaller, although pruning still removes four nodes from the selected 40-node configuration.

Scope of the claim: “Smallest good model” means the smallest model found among the tested candidates under the stated validation rule; it is not a proof of global minimality.

3. Interpretations and Conclusions

Main conclusion

The notebook demonstrates a functioning tree-type neural network in which small learned correction networks can deliberately drive an under-fit CNN to near-perfect training interpolation on two image datasets. The same mechanism that

produces the desired overfitting also reveals a generalisation cost, making the tree a useful experimental instrument rather than merely an accuracy booster.

3.1 A coherent interpretation of the architecture

The most important architectural choice is to give the tree a representation that is shared in origin but not necessarily identical at every node. The CNN trunk supplies progressively deeper feature summaries, while the correction nodes select combinations of these summaries according to their role and depth. This allows the tree to operate as a residual correction system over a common learned representation rather than as a collection of independent classifiers trained from raw pixels.

The varied feature strategy is especially informative because it performs competitively with the shared baseline while using fewer child parameters in the selected configuration. For Cats-vs-Dogs, the varied [16,32] tree achieves 99.99% train and 88.17% validation accuracy with 22,858 tree parameters, compared with 31,050 tree parameters for the shared [16,32] configuration. For CIFAR-10, the varied model uses 160,166 tree parameters versus 232,870 for the shared version, while both interpolate the training set.

Dataset	Configuration	Tree params	Nodes	Validation
CD	shared L2 w16	31,050	5	87.58%
CD	varied L2 w16	22,858	5	88.17%
C10	shared L2 w16	232,870	40	83.38%
C10	varied L2 w16	160,166	40	83.11%

The choice of varied routing is therefore not simply about maximising raw validation accuracy: the final selection uses a one-standard-error rule and prefers the lower-parameter configuration when its validation performance is statistically close enough to the best candidate.

3.2 What the overfitting experiment says

The tree clearly learns the training residual structure left by the under-fit CNN. The strongest evidence is not the final 99.99%/99.95% training accuracy by itself, but the fixes-versus-breaks accounting: almost every root error is corrected on the training set, whereas on clean test data there are more broken decisions than fixed decisions. This makes the generalisation failure mechanistically interpretable: the tree is learning highly specific corrections to the training examples.

The sign-test evidence is correspondingly extreme on training: $p = 7.39\text{e-}127$ for Cats-vs-Dogs and $p = 5.31\text{e-}266$ for CIFAR-10. On test data the net changes are negative (CD -31; CIFAR-10 -120), with $p = 0.00956$ and $1.05\text{e-}06$ respectively.

3.3 What the double-descent experiment says

The width sweep is most persuasive when read as a qualitative phenomenon. At small widths the models cannot fit the noisy labels. Near interpolation, clean-test error develops a hump. At larger widths, the models still fit the noisy labels but test error decreases substantially. This is the familiar interpolation-regime pattern the assignment asked us to look for.

The evidence is particularly coherent for CNN+tree on CIFAR-10 because all three seeds support the same rise-and-fall pattern. Cats-vs-Dogs is weaker but still suggestive with two of three seeds. The CNN-only baselines are less convincing, which makes the tree component itself an interesting part of the observed interpolation dynamics.

3.4 What the input-sensitivity maps suggest

The node maps are spatially non-uniform: the most sensitive 10% of cells hold far more than the 9.4% mass expected under uniform sensitivity. Child maps also retain some relationship to the root, but the relationship is dataset-dependent. Cats-vs-Dogs shows a relatively high mean child/root correlation (0.74), whereas CIFAR-10 is lower (0.50), suggesting stronger specialisation in the sampled CIFAR-10 correction nodes.

A useful conceptual interpretation is that the tree is not learning the original classification problem from scratch at every node. Instead, it starts from a common root representation and learns specialised functions that target particular failure modes. The child nodes therefore act more like learned residual detectors/correctors than independent full classifiers.

4. Future Directions and Extensions

The following directions would naturally extend the current experiment while preserving its central idea. They are ordered from the most immediate scientific next step to broader architectural questions.

1. Handcrafted features versus the learned CNN representation

This version tests whether a Tree-Type NN can learn corrections from engineered descriptors rather than learned convolutional features. Images use colour-grid appearance, coarse and fine HOG for shape, Sobel/Laplacian responses for detail, and LBP texture on Cats-vs-Dogs; a raw-pixel PCA representation is reserved for deeper nodes if the training target is missed. Nodes receive role-specific feature views, while CIFAR-10 uses ECOC to combine binary trees into a 10-class predictor. The handcrafted pipeline reaches roughly 70% test accuracy on Cats-vs-Dogs and 50% on CIFAR-10. By contrast, the final model learns multi-scale shallow, mid, and deep features using a shared CNN trunk, with small MLPs for residual corrections, reaching about 87.2% and 81.7% test accuracy, respectively.

2. Better isolate the source of correction features

Compare the shared-trunk design against partially or fully separate trunks for different branches. This would directly test whether branch-specific representation learning improves correction quality enough to justify the additional parameters.

3. Make the receptive-field study more spatially faithful

Replace global pooling with spatial feature heads or retain low-resolution spatial maps in selected nodes. This would let the correction MLPs encode explicit location information and make their learned sensitivity patterns easier to interpret.

4. Scale the double-descent experiment

Use more random seeds, a denser width sweep around interpolation, a larger data subset, and multiple noise levels. The present experiment is already suggestive; this extension would quantify the transition more robustly.

5. Optimise compute, not only parameter count

Measure FLOPs, inference latency, activation memory, and number of node evaluations. A tree with slightly more parameters but fewer active nodes per sample could be preferable in deployment.

6. Explore structured pruning and early exits

Because each branch corresponds to a correction task, nodes could be pruned or skipped conditionally at inference time. This creates a natural route toward input-adaptive computation.

7. Replace heuristic feature routing with learned routing

The current shallow/mid/deep policy is hand-designed. A gating mechanism could learn which feature level is most useful for each correction node while retaining a strict parameter budget.

8. Move beyond one-vs-rest multiclass trees

A single multiclass tree could share correction structure across classes instead of maintaining ten one-vs-rest trees. This may reduce redundant representations while making the hierarchy more interpretable.

9. Study regularisation of the tree itself

The same tree that produces 100% training interpolation could be regularised with node dropout, weight penalties, margin constraints, or validation-aware stopping to study whether some of the memorisation can be converted into generalisation.

4.1 Scope and limitations

Dataset scale: The main experiments use 6,800/1,200/2,000 Cats-vs-Dogs train/validation/test images and 12,750/2,250/10,000 CIFAR-10 train/validation/test images. These are subset experiments, not full-scale exhaustive training runs.

Double descent: The sweep uses three seeds and a heuristic verdict. The result should be called empirical evidence consistent with double descent, not proof of a universal law.

Receptive fields: The occlusion maps measure effective sensitivity to perturbations, not exact theoretical receptive fields or causal influence.

Size minimisation: The reported minimum is conditional on the finite candidate set explored by the notebook and the stated one-standard-error validation criterion.

Generalisation interpretation: The tree is intentionally trained to interpolate training residuals, so a larger train-test gap is an expected consequence of the experimental design rather than an implementation defect.

5. Reproducibility and Code

Private repository

GitHub repository containing code, documentation, and supporting material:

<https://github.com/Arnav2304-rgb/ELL409-Tree-Type-Neural-Network-Assignment.git>

The notebook is organised into twelve explicit steps, with resumable caches and CSV/JSON outputs written during the long sweeps. The final comparison and evidence tables are generated directly from measured quantities, reducing reliance on manually transcribed numbers.

Requirement coverage

Requirement	Notebook evidence	Assessment
Small neural-network nodes on both datasets	MLP correction nodes; binary CD tree and ten-tree CIFAR-10 one-vs-rest ensemble	Architecture + final results
Clever feature construction	Shared CNN trunk; shallow/mid/deep feature views; varied role/depth routing	Explained as central design choice
Overfit to ~100% training accuracy	99.99% CD and 99.95% CIFAR-10; growth curves	Train/test gaps and fixes/breaks analysed
Try double descent	20% noisy labels, width sweep, 3 seeds, clean test	Evidence and appropriate caveat provided
Receptive fields	Occlusion sensitivity maps and quantitative concentration/correlation statistics	Interpretation distinguishes effective sensitivity from strict theoretical RF
Minimise network size	Narrower trunks, node-size grid, pruning, Pareto analysis	Smallest-good models and limitations reported
Future directions / innovation	Multi-scale routing, pruning, interpolation study, adaptive corrections	Concrete next steps proposed

Closing perspective. The strongest contribution of the work is not a single accuracy number. It is the controlled experimental picture: start from an under-fit representation, grow small correction networks that interpolate the residuals, observe how this changes generalisation and model-size tradeoffs, and then inspect where individual correction nodes derive their signal. The architecture, experiments, and analysis are therefore aligned around one central scientific question: how far can a structured tree of small neural corrections push an intentionally imperfect CNN, and what do we learn from the capacity required to do so?